Java OOP Assignment

Build & Refactor a Leave Management System

Submitted by: Gurucharan Maripala

Due date: 28 October 2026

GitHub repository: [paste your repository link here]

https://github.com/MGurucharan/Leave_Managment_System_Gurucharan_Mariapala

1. Objective and Scenario

The aim is to build a simple Java application for a company that stores employee information, manages leave requests and applies different leave rules by employee type, and then to improve it step by step using Object-Oriented Programming principles.

Employee type	Annual leave	Max days per request
Full-time	20	10
Part-time	10	5
Intern	5	2
Manager (change request)	30 (20 + 10 bonus)	15

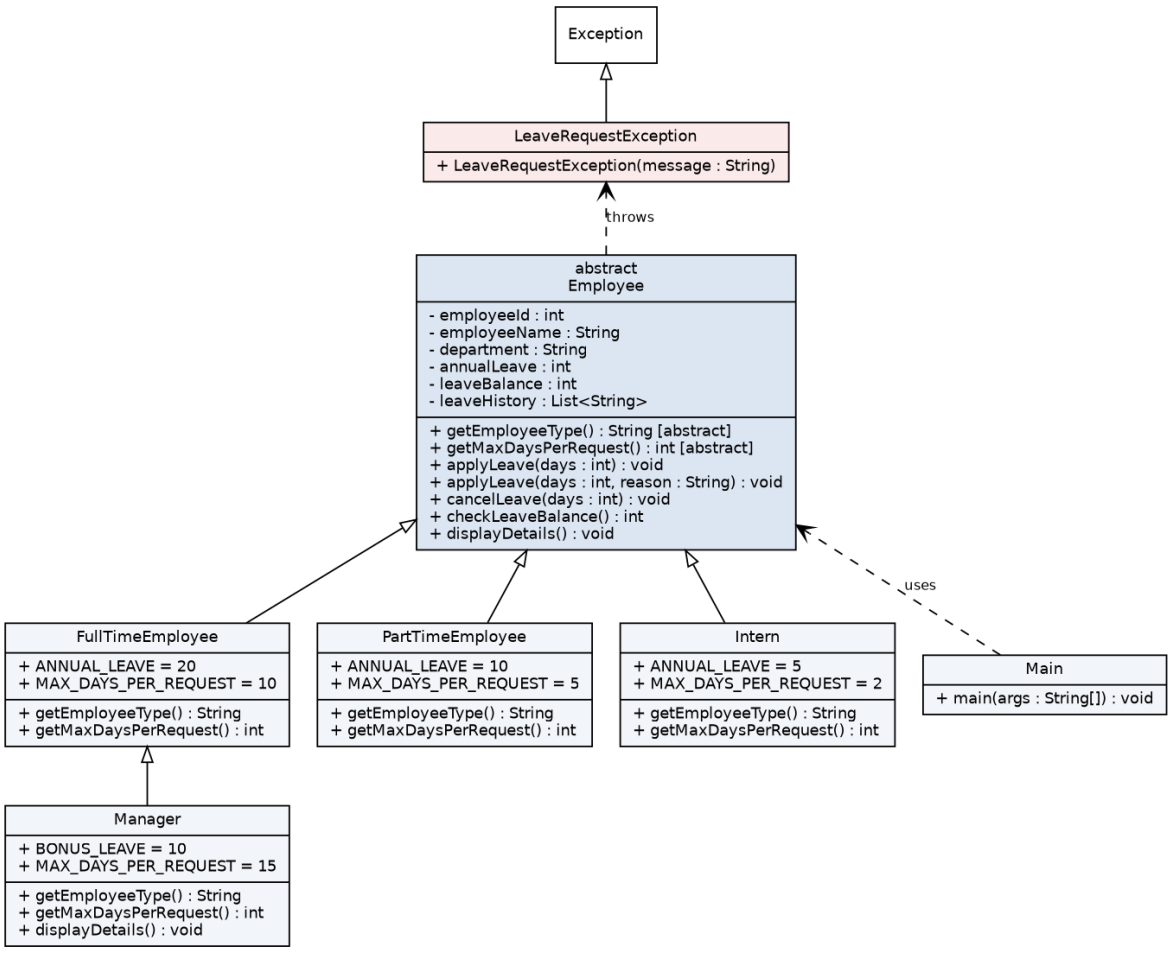
2. Design Before Coding

Main objects: Employee and its types (FullTimeEmployee, PartTimeEmployee, Intern, Manager) and LeaveRequestException.

Data: employeeId, employeeName, department, annualLeave, leaveBalance, leaveHistory.
Behaviours: applyLeave(), cancelLeave(), displayDetails(), checkLeaveBalance().

Relationships: the three employee types inherit from Employee; Manager inherits from FullTimeEmployee; Employee throws LeaveRequestException.

Class diagram



3. Implementation Summary

The application was built in stages and committed to Git after each stage:

Commit	Stage
1. Initial employee design	Objects, data, behaviours and relationships documented
2. Added Employee class	Fields, constructor, getters, displayDetails()
3. Implemented leave management	applyLeave(), cancelLeave(), checkLeaveBalance()
4. Added inheritance	FullTimeEmployee, PartTimeEmployee, Intern extend Employee
5. Implemented polymorphism	Abstract Employee, overriding, overloading, polymorphic array in Main
6. Added Manager	Change request: extra leave benefits
7. Refactored code	Removed duplication, added exceptions, constants, better encapsulation
8. Final version	README, class diagram, sample output, notes

4. Explanation of OOP Concepts Used

Concept	Where	How
Encapsulation	Employee	Fields are private; the leave balance can only change through applyLeave() and cancelLeave(), which validate input. History is returned as an unmodifiable list.
Inheritance	FullTimeEmployee, PartTimeEmployee, Intern extend Employee; Manager extends FullTimeEmployee	Subclasses reuse all leave logic and only add what is different.
Polymorphism	Main	One Employee[] holds every type; calling applyLeave() or displayDetails() runs the correct rules for each object at runtime.
Abstraction	Employee is abstract	getEmployeeType() and getMaxDaysPerRequest() are abstract, so every employee type must define its own rules, and Employee cannot be instantiated.
Method overriding	getEmployeeType(), getMaxDaysPerRequest(), Manager.displayDetails()	Subclasses replace or extend parent behaviour.
Method overloading	Employee.applyLeave(int) and applyLeave(int, String)	Same name, different parameters.
Access modifiers	private, protected, public	private for data, protected constructors so only subclasses can build base types, public for the API.

5. Change Request: Manager

A Manager type with additional leave benefits was added. It receives 10 bonus days on top of the full-time entitlement (30 in total) and may take up to 15 days per request. In the first version Manager copied the full-time logic (see the *before-refactor* tag). After refactoring, Manager extends FullTimeEmployee and only defines what is different, which keeps the design reusable and maintainable.

6. Debugging and Refactoring Notes

#	Problem found	Fix
1	applyLeave() copied into four subclasses (duplicate code)	Logic moved once into Employee; subclasses only supply getMaxDaysPerRequest()
2	Methods returned boolean and printed errors, mixing logic with output	Custom LeaveRequestException with clear messages; Main handles display
3	leaveBalance was protected and could bypass rules	Made private; access only through the public API
4	Magic numbers and unclear names such as d	Named constants and clear variable names
5	Manager duplicated full-time rules	Manager now extends FullTimeEmployee
6	cancelLeave() could exceed entitlement; blank names accepted	Added validation checks and a leave history

7. Sample Output

```
=== Employee details ===
Full-time  | ID: 1 | Name: Alice  | Dept: HR         | Balance: 20/20 day(s)
Part-time  | ID: 2 | Name: Bob    | Dept: IT         | Balance: 10/10 day(s)
Intern     | ID: 3 | Name: Carol  | Dept: Finance    | Balance:  5/5 day(s)
Manager    | ID: 4 | Name: David  | Dept: Operations | Balance: 30/30 day(s)
           (includes 10 bonus leave day(s) for managers)

=== Valid leave requests ===
OK   : Alice applied 5 day(s)
OK   : Bob applied 3 day(s)
OK   : Carol applied 2 day(s)
OK   : David applied 14 day(s)

=== More requests (rules enforced per employee type) ===
ERROR: Bob - Part-time can take at most 5 day(s) per request
ERROR: Carol - Intern can take at most 2 day(s) per request
ERROR: Alice - Number of days must be greater than zero
OK   : Carol applied 2 day(s)
ERROR: Carol - Not enough leave balance (available: 1)

=== Cancelling leave ===
OK   : Alice cancelled 2 day(s)
ERROR: Alice - Cannot cancel more leave than was taken

=== Final balances ===
Alice (Full-time): 17 day(s) left
Bob (Part-time): 7 day(s) left
Carol (Intern): 1 day(s) left
David (Manager): 16 day(s) left

=== Alice's leave history ===
 - Applied 5 day(s) - Family event
 - Cancelled 2 day(s)
```

8. Source Code

Employee.java

```
package leavemgmt;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Abstract base class for every employee type.
 * Common leave logic lives here; subclasses only supply their own rules.
 */
public abstract class Employee {
    private final int employeeId;
    private final String employeeName;
    private final String department;
    private final int annualLeave;
    private int leaveBalance;
    private final List<String> leaveHistory = new ArrayList<>();

    protected Employee(int employeeId, String employeeName, String department, int annualLeave) {
        if (employeeName == null || employeeName.isBlank()) {
            throw new IllegalArgumentException("Employee name must not be empty");
        }
        if (annualLeave < 0) {
            throw new IllegalArgumentException("Annual leave must not be negative");
        }
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.department = department;
        this.annualLeave = annualLeave;
        this.leaveBalance = annualLeave;
    }

    // ---------- Abstract methods: each employee type defines its own rules ----------
    public abstract String getEmployeeType();

    public abstract int getMaxDaysPerRequest();

    // ---------- Getters (encapsulation: fields stay private) ----------
    public int getEmployeeId() { return employeeId; }
    public String getEmployeeName() { return employeeName; }
    public String getDepartment() { return department; }
    public int getAnnualLeave() { return annualLeave; }
    public List<String> getLeaveHistory() { return Collections.unmodifiableList(leaveHistory); }

    public int checkLeaveBalance() {
        return leaveBalance;
    }

    // ---------- Method overloading: same name, different parameters ----------
    public void applyLeave(int days) throws LeaveRequestException {
        applyLeave(days, "Not specified");
    }

    public void applyLeave(int days, String reason) throws LeaveRequestException {
        validateDays(days);
        if (days > getMaxDaysPerRequest()) {
            throw new LeaveRequestException(getEmployeeType() + " can take at most "
                    + getMaxDaysPerRequest() + " day(s) per request");
        }
        if (days > leaveBalance) {
            throw new LeaveRequestException("Not enough leave balance (available: " + leaveBalance + ")");
        }
        leaveBalance -= days;
        leaveHistory.add("Applied " + days + " day(s) - " + reason);
    }

    public void cancelLeave(int days) throws LeaveRequestException {
        validateDays(days);
        if (leaveBalance + days > annualLeave) {
```

```
            throw new LeaveRequestException("Cannot cancel more leave than was taken");
        }
        leaveBalance += days;
        leaveHistory.add("Cancelled " + days + " day(s)");
    }

    // Subclasses may override to add more details (method overriding)
    public void displayDetails() {
        System.out.printf("%-10s | ID: %d | Name: %-6s | Dept: %-10s | Balance: %2d/%d day(s)%n",
                getEmployeeType(), employeeId, employeeName, department, leaveBalance, annualLeave);
    }

    private void validateDays(int days) throws LeaveRequestException {
        if (days <= 0) {
            throw new LeaveRequestException("Number of days must be greater than zero");
        }
    }
}
```

FullTimeEmployee.java

```
package leavemgmt;

public class FullTimeEmployee extends Employee {
    public static final int ANNUAL_LEAVE = 20;
    public static final int MAX_DAYS_PER_REQUEST = 10;

    public FullTimeEmployee(int id, String name, String department) {
        this(id, name, department, ANNUAL_LEAVE);
    }

    // Lets subclasses (e.g. Manager) reuse this class with a different entitlement
    protected FullTimeEmployee(int id, String name, String department, int annualLeave) {
        super(id, name, department, annualLeave);
    }

    @Override
    public String getEmployeeType() {
        return "Full-time";
    }

    @Override
    public int getMaxDaysPerRequest() {
        return MAX_DAYS_PER_REQUEST;
    }
}
```

PartTimeEmployee.java

```
package leavemgmt;

public class PartTimeEmployee extends Employee {
    public static final int ANNUAL_LEAVE = 10;
    public static final int MAX_DAYS_PER_REQUEST = 5;

    public PartTimeEmployee(int id, String name, String department) {
        super(id, name, department, ANNUAL_LEAVE);
    }

    @Override
    public String getEmployeeType() {
        return "Part-time";
    }

    @Override
    public int getMaxDaysPerRequest() {
        return MAX_DAYS_PER_REQUEST;
    }
}
```

Intern.java

```
package leavemgmt;
public class Intern extends Employee {
```

```
    public static final int ANNUAL_LEAVE = 5;
    public static final int MAX_DAYS_PER_REQUEST = 2;

    public Intern(int id, String name, String department) {
        super(id, name, department, ANNUAL_LEAVE);
    }

    @Override
    public String getEmployeeType() {
        return "Intern";
    }

    @Override
    public int getMaxDaysPerRequest() {
        return MAX_DAYS_PER_REQUEST;
    }
}
```

Manager.java

```
package leavemgmt;

/**
 * Change request: Manager gets extra leave benefits.
 * Reuses FullTimeEmployee, so only the differences are written here.
 */
public class Manager extends FullTimeEmployee {
    public static final int BONUS_LEAVE = 10;
    public static final int MAX_DAYS_PER_REQUEST = 15;

    public Manager(int id, String name, String department) {
        super(id, name, department, FullTimeEmployee.ANNUAL_LEAVE + BONUS_LEAVE);
    }

    @Override
    public String getEmployeeType() {
        return "Manager";
    }

    @Override
    public int getMaxDaysPerRequest() {
        return MAX_DAYS_PER_REQUEST;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("           (includes " + BONUS_LEAVE + " bonus leave day(s) for managers)");
    }
}
```

LeaveRequestException.java

```
package leavemgmt;

/** Thrown when a leave operation breaks a business rule. */
public class LeaveRequestException extends Exception {
    private static final long serialVersionUID = 1L;

    public LeaveRequestException(String message) {
        super(message);
    }
}
```

Main.java

```
package leavemgmt;

public class Main {
    public static void main(String[] args) {
        // Polymorphism: one array, many employee types
        Employee[] staff = {
            new FullTimeEmployee(1, "Alice", "HR"),
            new PartTimeEmployee(2, "Bob", "IT"),
```

```
            new Intern(3, "Carol", "Finance"),
            new Manager(4, "David", "Operations")
        };

        System.out.println("=== Employee details ===");
        for (Employee e : staff) {
            e.displayDetails();
        }

        System.out.println("\n=== Valid leave requests ===");
        tryApply(staff[0], 5, "Family event");
        tryApply(staff[1], 3, "Medical appointment");
        tryApply(staff[2], 2, "Exam preparation");
        tryApply(staff[3], 14, "Annual holiday");

        System.out.println("\n=== More requests (rules enforced per employee type) ===");
        tryApply(staff[1], 6, "Trip");       // part-time max is 5
        tryApply(staff[2], 3, "Vacation");   // intern max is 2
        tryApply(staff[0], 0, "Test");       // invalid days
        tryApply(staff[2], 2, "Second request");
        tryApply(staff[2], 2, "Third request");  // balance too low

        System.out.println("\n=== Cancelling leave ===");
        tryCancel(staff[0], 2);
        tryCancel(staff[0], 10);             // more than was taken

        System.out.println("\n=== Final balances ===");
        for (Employee e : staff) {
            System.out.println(e.getEmployeeName() + " (" + e.getEmployeeType() + "): "
                    + e.checkLeaveBalance() + " day(s) left");
        }

        System.out.println("\n=== Alice's leave history ===");
        staff[0].getLeaveHistory().forEach(h -> System.out.println(" - " + h));
    }

    private static void tryApply(Employee e, int days, String reason) {
        try {
            e.applyLeave(days, reason);
            System.out.println("OK   : " + e.getEmployeeName() + " applied " + days + " day(s)");
        } catch (LeaveRequestException ex) {
            System.out.println("ERROR: " + e.getEmployeeName() + " - " + ex.getMessage());
        }
    }

    private static void tryCancel(Employee e, int days) {
        try {
            e.cancelLeave(days);
            System.out.println("OK   : " + e.getEmployeeName() + " cancelled " + days + " day(s)");
        } catch (LeaveRequestException ex) {
            System.out.println("ERROR: " + e.getEmployeeName() + " - " + ex.getMessage());
        }
    }
}
```

9. How to Run

```
javac -d out src/leavemgmt/*.java
java -cp out leavemgmt.Main
```